

User Manual: Ontology-Based Repair Procedure Application

Overview

This manual explains how to use the Repair Procedure Ontology app, how the underlying schema and ontology work, and how to query and manipulate the knowledge graph.

Application Structure

The app is built using Flask and Owlready2 for loading, managing, and querying the ontology. The interface allows users to:

1. Browse repair procedures and items related to vehicles.
2. Search for procedures by various criteria, including titles and tools used.
3. Identify and address data inconsistencies.
4. Query the ontology using predefined rules and view inferred relationships between items.

Ontology Schema

Main Classes:

- Item: Represents items such as tools or vehicle parts.
 - Subclasses: Tool, Part
- Procedure: Represents repair procedures for specific items.
- Step: Represents a specific step within a procedure.
- Image: Links images to steps for visual instructions.

Key Object Properties:

- has_procedure: Connects an item with its repair procedure.

- `has_step`: Connects a procedure with individual steps.
- `has_tool`: Links tools used in each step.
- `part_of`: A transitive property representing hierarchical relationships between items (e.g., parts that are part of larger assemblies).
- `issubProcedure`: Denotes if one procedure is a subprocedure of another.

Running the Application

Prerequisites:

1. Install **Python 3.x**.
2. Install dependencies using pip:

`pip install flask owlready2`

Starting the App:

1. Navigate to the directory containing the `app.py` file.
2. Run the Flask app:
`python app.py`
3. Open your browser and go to **`http://127.0.0.1:5000/`**.

Application Usage

Browsing Items and Procedures:

1. On the homepage, you'll see a list of items with their associated procedures and any inferred relationships (e.g., `part_of`).
2. You can filter items by the alphabet or the number of procedures using the buttons provided.
3. You can also search for procedures by typing in the search bar.

Viewing Procedure Details:

1. Click on a procedure title to view its details.
2. The procedure detail page lists:
 - Tools Required: Tools needed for the procedure.
 - Steps: Detailed steps for the procedure, along with any images.
 - Subprocedure Links: Links to subprocedures (if any).

Querying the Ontology

Example Queries

Query 1: Procedures with More than 6 Steps

This query retrieves procedures that contain more than 6 steps.

```
SELECT ?procedure (COUNT(?step) AS ?stepCount)
```

```
WHERE {
```

```
    ?procedure rdf:type ex:Procedure .
```

```
    ?procedure ex:has_step ?step .
```

```
}
```

```
GROUP BY ?procedure
```

```
HAVING (COUNT(?step) > 6)
```

Query 2: Items with More than 10 Procedures

```
SELECT ?item (COUNT(?procedure) AS
```

```
?procedureCount) WHERE {
```

`?item rdf:type ex:Item .`

`?item ex:has_procedure ?procedure .`

`}`

`GROUP BY ?item`

`HAVING (COUNT(?procedure) > 10)`

How to Form Queries

1. Queries should follow the SPARQL syntax.
2. Use prefixes to shorten common URIs, such as ex: for the example namespace http://example.org/repair_ontology.owl#.
3. Use FILTER to narrow down results based on certain properties (e.g., keywords in step titles).

Modifying the Ontology

Adding New Data

Open the Vehicle.json file and add a new item or procedure in the JSON format.
For example, to add a new procedure for an item:

```
{ "Title": "Example Procedure", "Guidid": 1234, "Category": "Example Category", "Toolbox":  
[{"Name": "Screwdriver", "Url": "/Item/Screwdriver"}], "Steps": [{"Order": 0, "Text_raw": "Step  
description", "Images": ["image_url"]}], "Url": "https://example.com/procedure" }
```

Updating Data

Modify the relevant entry in Vehicle.json and run the ontology creation script again.

Deleting Data

Remove the entry from Vehicle.json and re-run the script to update the knowledge graph.

Adding Rules to the Ontology

To add new rules, modify the ontology structure in app.py. For example, to add a new rule that checks for a certain relationship between tools and steps:

```
class Step(Thing):  
    equivalent_to = [  
        Thing & has_tool.only(Tool) & has_tool.some(has_tool.some(Tool))  
    ]
```

After adding a rule, make sure to re-run the reasoning process:

```
sync_reasoner()
```

Troubleshooting and Error Detection

The app checks for missing tools and steps in procedures and flags these in the UI. If the data is incomplete or inconsistent, warnings will appear next to the procedure name.

Future Extensions

New Queries: Users can add custom SPARQL queries to further explore the knowledge graph.

Custom Inferences: Additional rules and constraints can be added to the ontology to improve the reasoning process.